

Getting Started

Running the Universal Audio Plugin Template in Claude Code

How to copy the workflow template, point an IDE (VS Code, Google Antigravity, etc.) at the project, and run the exact first-session script — Phase 0 through the start of Phase 1B — using Claude Code.

What Each Phase Does

The steps below only walk you through Phase 0 through the start of Phase 1B. Every phase after that follows the same three-message rhythm (Step 5). Here's the whole journey in plain terms first — the full technical roadmap (skills + documents each phase produces) is on the next page.

0 — Project Setup	Make sure the tools you need are installed - nothing is built yet.
1A — DSP Design	Decide what the plugin should sound like and how it works - just planning, no code.
1B — Software Architecture	Sketch out how the code will be organized before writing any of it.
2 — CMake + JUCE Skeleton	Set up the empty project so it builds and opens in a DAW.
3 — Parameter Architecture	List every knob and switch the plugin will have.
4 — DSP Implementation	Write the actual code that processes the audio.
5 — Advanced DSP	Add trickier features on top - like oversampling or MIDI.
6 — UI Design (HTML)	Design what the plugin looks like, first as a quick web mockup.
7 — UI Implementation	Build the real on-screen interface from that mockup.
8 — Preset System	Let users save and load their own settings.
9 — Optimization	Make the plugin run faster without changing how it sounds.
10 — QA & DAW Validation	Test the plugin thoroughly in real DAWs.
B — Beta Testing	Let real users try it and give feedback before release.
R — Release & Distribution	Package it up and ship the finished plugin.

Phase Roadmap — Skills and Documents

This table is the fixed process from `CLAUDE.md § 3` and never changes. State Doc is the file in `Docs/State/` that phase reads from and writes into — see `CLAUDE.md § 8` for the full protocol.

Phase	Name	Skill	State Doc
0	Project Setup	<i>no skill — CLAUDE.md § 6 checklist</i>	—
1A	DSP Design	<i>dsp-research</i>	DSP_Design.md
1B	Software Architecture	<i>plugin-architecture</i>	Software_Architecture.md
2	CMake + JUCE Skeleton	<i>build-system</i>	Build_Config.md
3	Parameter Architecture	<i>plugin-architecture</i>	Parameters.md
4	DSP Implementation	<i>dsp-implementation</i>	DSP_Implementation.md
5	Advanced DSP	<i>dsp-implementation</i>	DSP_Implementation.md
6	UI Design (HTML)	<i>plugin-ui</i>	UI_Design.md
7	UI Implementation	<i>plugin-ui</i>	UI_Implementation.md
8	Preset System	<i>plugin-architecture</i>	Presets.md
9	Optimization	<i>plugin-optimization</i>	Build_Config.md
10	QA & DAW Validation	<i>plugin-qa</i>	—
B	Beta Testing	<i>no skill — Docs/BetaTesting.md</i>	—
R	Release & Distribution	<i>plugin-release</i>	—

***B** and **R** are letters, not numbers — Beta Testing and Release & Distribution run after Phase 10's hard blockers pass and are the only two phases without a numeric slot. `ChangeLog.md` isn't listed per-row because every phase appends to it regardless.*

Step 1 — Copy the template and rename it

Make a copy of the `plugin-workflow` template folder into the location where your plugin project will live, and rename the copy to your plugin's name. Claude Code discovers `CLAUDE.md` and `.claude/skills/` based on the current working directory, so they need to live at the project root — not nested inside another folder.

```
cp -R ~/DEV/plugin-workflow ~/DEV/PluginName
cd ~/DEV/PluginName
```

Prefer Finder? Duplicate the template folder (Cmd+D) and rename it instead of using the terminal — same result. `.claude/` is a hidden folder (dot prefix); use Cmd+Shift+. in Finder to see it and confirm it copied over.

After this, `CLAUDE.md`, `.claude/skills/`, `Docs/`, and `Tests/` should sit directly inside `~/DEV/PluginName/` — not nested inside another folder.

Step 2 — Fill in the two documents everything else depends on

Do this in a text editor, before opening Claude Code. It's tedious form-filling, not something worth burning a conversation on.

1. `Docs/PluginSpec.md` — every field. This is the single document every skill reads from. Leaving it half-empty means every phase downstream will ask you the same questions back.
2. `CLAUDE.md § 1` — the project identity table (plugin name, manufacturer code, JUCE path, project root, etc).

Note: Skip `Docs/State/*.md` for now. Those are empty templates meant to fill in during Phase 1, not before.

Step 3 — Open the IDE in that exact folder

In VS Code or Antigravity: File → Open Folder → select `~/DEV/PluginName`. Then open the Claude Code panel or terminal from inside that workspace.

Note: If Claude Code's working directory isn't the project root, it will not find `CLAUDE.md` or `.claude/skills/` at all, and the entire routing system will silently do nothing. This is the most common first-session failure.

Step 4 — Run the first-session script

Send these messages one at a time, in order. Read each response before sending the next — don't batch them.

Message 1 — Sanity check — confirms Claude Code can see your files

```
Read CLAUDE.md and confirm you have it. List the phase roadmap from § 3.  
Then read Docs/PluginSpec.md and tell me which fields are still empty.  
Do not begin any implementation.
```

If this fails (Claude says it can't find the file): your working directory is wrong. Check with `pwd` in the Claude Code terminal — it must be the project root.

Message 2 — Phase 0 — toolchain check

```
Let's go through the Phase 0 checklist in CLAUDE.md § 6.  
Check what you can verify yourself via the shell (cmake --version,  
xcode-select -p, git --version, JUCE folder existence at the path I gave  
in CLAUDE.md § 1). For anything you can't verify yourself, ask me directly.
```

Answer whatever it asks. If JUCE isn't installed yet, install it now before continuing — Phase 2 needs it.

Message 3 — Close out Phase 0

```
Phase 0 complete. Update CLAUDE.md § 2 current phase to:  
Phase 1A / NOT STARTED.
```

Small thing, but it matters — this is the one stateful line in the whole system, and it should be accurate from the start.

Message 4 — Start Phase 1A — DSP research

```
Phase 1A starting. Load .claude/skills/dsp-research/SKILL.md.  
Read Docs/PluginSpec.md § 2.2, 2.3, 2.7.  
Begin Phase 1A — produce the DSP Design Document. Do not write any code.
```

Claude will produce the design document inline in chat. Read it carefully — push back on anything that doesn't match your sonic intent before approving. This is the cheapest point in the whole project to catch a wrong assumption.

Message 5 — Approve and commit to state

```
Approved. Write this into Docs/State/DSP_Design.md, replacing the
placeholder content section by section. Append one line to
Docs/State/Changelog.md per the format in CLAUDE.md § 8.
```

Message 6 — Phase 1B — architecture

```
Phase 1B starting. Load .claude/skills/plugin-architecture/SKILL.md.
Read Docs/State/DSP_Design.md.
Produce the Software Architecture Document. No code yet.
```

Same pattern as before — review, then approve, then:

```
Approved. Write this into Docs/State/Software_Architecture.md.
Append to Docs/State/Changelog.md.
Update CLAUDE.md § 2 current phase to: Phase 2 / NOT STARTED.
```

Step 5 — The repeating rhythm for every phase after this

From here on, every phase follows the same three-message pattern. The phase-to-skill mapping always lives in CLAUDE.md § 3 if you forget which skill goes with which phase — you can ask Claude “what’s next” and it will read the table itself.

1. Phase N starting. Load .claude/skills/[skill-name]/SKILL.md. Read [relevant Docs/State files].
2. Work through it, review, push back where needed.
3. Approved. Write into Docs/State/[Domain].md. Append to Changelog. Update CLAUDE.md § 2.

One practical habit

Start a new Claude Code conversation per phase, not one continuous thread. The Docs/State/ files are what carry context between sessions — that’s the whole point of replacing handoffs with them. A fresh conversation that reads Docs/State/*.md will have cleaner context than a single long thread accumulating drift.

Note: Claude Code does not auto-load skill files just because they exist in .claude/skills/. You must explicitly tell it to read one the first time in a session (“Load .claude/skills/dsp-research/SKILL.md”). If a session feels like it’s ignoring the rules, check whether the skill was actually read into context that conversation.